

Project Documentation

Full Stack Development – Project Documentation Format

Adapted for a ServiceNow project (the template's React, Node.js and MongoDB items are replaced by their ServiceNow equivalents).

1. Introduction

- **Project Title:** Script-Controlled ACL – Restrict Record Access Based on Field Value
- **Team ID:** SWTID-2026-1268
- **Team Members:** *[add team members and roles]*

2. Project Overview

Purpose: Restrict access to records based on the value of a field, using a script-controlled ACL, so users see only the records they are permitted to see.

Features:

- Role creation and assignment; login with email/password or SSO.
- ACL on a table and controlling field, with field-value conditions.
- ACL scripts that return true/false using the record field value and current user.
- Filtered list views, protection against direct URL and API access, and field masking.
- Clear access denied message and an audit log with report export.

3. Architecture

- **Frontend (in place of React):** ServiceNow Next Experience UI and Service Portal – lists, forms and portal pages, with client scripts and UI policies where needed.
- **Backend (in place of Node.js / Express.js):** server-side JavaScript on the platform – ACL scripts, script includes and business rules that run on every request.
- **Database (in place of MongoDB):** ServiceNow relational tables, accessed through GlideRecord and the platform's REST Table API.

Database schema (example):

Table	Key fields	Purpose
u_employee_case (custom)	number, short_description, u_department (reference), assigned_to, u_notes (sensitive)	Records protected by the ACL
sys_user	user_name, department, active	Users and their attributes
sys_user_has_role	user, role	Role assignments
sys_security_acl	name, operation, script, active	ACL rules and scripts
Audit / system log	user, record, time, result	Allowed and denied attempts

Interaction: the ACL script reads u_department from the record and the user's department/roles, and returns true or false before the platform returns any data.

4. Setup Instructions

Prerequisites

- A ServiceNow Personal Developer Instance (free from the ServiceNow Developer program) with administrator access.

- A modern web browser; no local Node.js or database is needed.

Installation

1. Request and sign in to a Personal Developer Instance.
2. Create an application scope in Studio (or use Global for testing).
3. Create the table `u_employee_case` with a Department reference field and a sensitive notes field.
4. Create roles (for example `hr_user`, `finance_user`, `auditor`) and test users, and assign roles.
5. Create an ACL on the table for the read operation and add the ACL script and field-value condition.
6. Add sample records for different departments.

5. Folder Structure

- **Client (instead of a React folder):** forms, lists, UI policies, client scripts and portal widgets of the application.
- **Server (instead of a Node.js folder):** tables, ACLs, script includes, business rules and scheduled jobs of the application.

6. Running the Application

There are no npm commands: the application runs on the hosted instance. Open the instance URL, then use **Impersonate User** to test as different users (for example an HR user and a Finance user) and open the table list and records.

7. API Documentation

Endpoints use the platform's REST Table API. ACLs are enforced on every call. Examples below are illustrative.

Method	Endpoint	Parameters	Description
GET	<code>/api/now/table/u_employee_case</code>	<code>sysparm_limit</code> , <code>sysparm_query</code>	Lists only the records the caller is permitted to read
GET	<code>/api/now/table/u_employee_case/{sys_id}</code>	<code>sys_id</code>	Returns one record, or an error if the ACL denies access
POST	<code>/api/now/table/u_employee_case</code>	JSON body with fields	Creates a record if the create ACL allows it

Example response (denied record):

```
{
  "error": {
    "message": "No Record found",
    "detail": "Record doesn't exist or ACL restricts the record retrieval"
  },
  "status": "failure"
}
```

8. Authentication

- **Authentication:** users log in with email/password or company SSO (SAML 2.0 / LDAP); the platform keeps a session after login.
- **Authorization:** roles control table-level access; the script-controlled ACL then decides access for each record from its field value. Access is denied by default, and script errors deny access.

9. User Interface

[Insert screenshot: list view for an HR user]

[Insert screenshot: record form for a permitted user]

[Insert screenshot: access denied message for a restricted record]

[Insert screenshot: ACL record and script]

10. Testing

- **Functional testing:** input validation, field-value conditions, ACL enforcement, API checks (FT-01 to FT-04).
- **Performance testing:** response time, concurrent API calls and large list loads (PT-01 to PT-03).
- **User acceptance testing:** 13 scenarios with positive and negative cases, plus bug tracking.
- **Tools:** Impersonate User, the ACL debugging tools and the platform's REST API explorer.

11. Screenshots or Demo

[Insert screenshot: end-to-end demo of a permitted and a denied access]

[Insert screenshot: access log report]

Demo link: *[add link]*

12. Known Issues

- Heavy queries inside ACL scripts can slow down large lists.
- Records with an empty controlling field are denied to everyone except administrators.
- Four early bugs found during UAT (direct URL access, empty field handling, message wording, log timestamp) were fixed; add any open issues here.

13. Future Enhancements

- No-code rule builder for field-value conditions.
- Alerts and dashboards for repeated denied attempts.
- Reusable policy templates (department, region, owner).
- Approval workflow for ACL script changes.